

MIRS Upload Guide: Redkite Subdomain and Git P

Generated 2026-06-02

This guide explains how to prepare the Material Inspection Recording System for upload to a z.com/cPanel subdomain under redkite.com.ph and how to store the source code in a Git repository.

NOTE: This is a Laravel application, not a WordPress site. Do not upload the whole project directly into public_html. Only Laravel public files should be web-accessible.

Part 1 - What This Project Is

MIRS is a Laravel 9 application with a Vue/Inertia frontend. It uses PHP, Composer, Node/Vite, MySQL, Laravel routes/controllers/models, and compiled frontend assets in public/build.

Because it is Laravel, the application has private server files such as app, config, database, routes, storage, vendor, and .env. These should not be publicly browseable.

Part 2 - Recommended z.com Folder Layout

Use one private folder for the Laravel application and one public folder for the subdomain document root.

```
/home/YOUR_CPANEL_USER/mirs_app/  
  app/  
  bootstrap/  
  config/  
  database/  
  public/  
  resources/  
  routes/  
  storage/  
  vendor/  
  .env  
  artisan  
  composer.json  
  composer.lock  
  
/home/YOUR_CPANEL_USER/public_html/mirs/  
  index.php  
  .htaccess  
  build/  
  favicon.ico  
  robots.txt
```

The subdomain, for example mirs.redkite.com.ph, should point to public_html/mirs or another folder that contains only the contents of Laravel public.

Part 3 - Files to Upload to Hosting

Upload these Laravel application files to the private app folder: app, bootstrap, config, database, lang, public, resources, routes, storage, vendor, artisan, composer.json, composer.lock, and .env.

Upload only the contents of public into the subdomain public folder. Include hidden files such as .htaccess.

Do not upload these to the public subdomain folder: .env, app, bootstrap, config, database, resources, routes, storage, vendor, node_modules, .git, tests, or temporary files.

Part 4 - Prepare the App Locally

Before uploading, build the frontend assets locally from the project folder:

```
npm install
npm run build
```

This creates production frontend assets inside public/build. Make sure public/build is included when uploading the public folder contents.

If Composer is available on the hosting server, install PHP dependencies on the server. If it is not available, upload the existing vendor folder from your local project.

```
composer install --no-dev --optimize-autoloader
```

Part 5 - Create the z.com Subdomain

- Log in to z.com and open cPanel.
- Go to Domains or Subdomains.
- Create a subdomain such as mirs.redkite.com.ph.
- Set the document root to public_html/mirs or the chosen public folder.
- If cPanel has a Share document root option, leave it unchecked.
- Enable SSL/HTTPS using AutoSSL, Let's Encrypt, or the SSL option available in cPanel.

Part 6 - Update public/index.php

After copying Laravel public files into the subdomain folder, edit the public index.php on the server so it loads Laravel from the private app folder.

Original Laravel paths usually look like this:

```
require __DIR__.'/../vendor/autoload.php';
$app = require_once __DIR__.'/../bootstrap/app.php';
```

For the recommended cPanel layout, change them to something like this:

```
require __DIR__.'/../../mirs_app/vendor/autoload.php';
$app = require_once __DIR__.'/../../mirs_app/bootstrap/app.php';
```

NOTE: The number of ../ segments depends on the actual folder path in cPanel. The important goal is that index.php points to vendor/autoload.php and bootstrap/app.php inside the private mirs_app folder.

Part 7 - Database Setup

- In cPanel, open MySQL Databases or MySQL Database Wizard.
- Create a database, database user, and strong password.

- Assign the user to the database with full privileges.
- Export the current local database from XAMPP phpMyAdmin.
- Import the SQL file into the z.com database using phpMyAdmin.

Part 8 - Server .env Settings

Create or edit the .env file in the private Laravel app folder, not in the public subdomain folder.

```
APP_NAME="Material Inspection Recording System"
APP_ENV=production
APP_DEBUG=false
APP_URL=https://mirs.redkite.com.ph

DB_CONNECTION=mysql
DB_HOST=localhost
DB_PORT=3306
DB_DATABASE=your_cpanel_database
DB_USERNAME=your_cpanel_database_user
DB_PASSWORD=your_database_password

SESSION_SECURE_COOKIE=true
```

Part 9 - Laravel Commands If SSH Is Available

If z.com provides SSH or Terminal access, run these from the private Laravel app folder after uploading and configuring .env:

```
php artisan key:generate
php artisan config:cache
php artisan route:cache
php artisan view:cache
php artisan storage:link
```

If SSH is not available, generate APP_KEY locally if needed, upload the prepared .env, and avoid running cache commands until you can access Terminal or SSH.

Part 10 - No-Index Safeguards

The current project includes no-index safeguards so search engines are instructed not to index the site.

- public/robots.txt blocks crawling with Disallow: /.
- The main Blade layout includes a robots meta tag with noindex, nofollow, noarchive, nosnippet.
- Laravel responses include X-Robots-Tag: noindex, nofollow, noarchive, nosnippet.
- public/.htaccess sets the same X-Robots-Tag header for Apache when mod_headers is enabled.

NOTE: No-index is not the same as private access. It discourages search indexing, but anyone with the URL can still reach the login page unless you add stronger access controls.

Part 11 - Git Repository Setup

Use Git for source code backup/version control. The Git repository should contain source code, configuration examples, and documentation, but not secrets or generated dependency folders.

Before committing, confirm .gitignore excludes sensitive/generated files such as .env, node_modules, vendor, public/build, public/storage, and storage/*.key.

```
git init
git status
git add .
git commit -m "Initial MIRS project upload"
```

Create a repository on GitHub, GitLab, Bitbucket, or your preferred Git hosting provider. Then connect and push:

```
git remote add origin https://github.com/YOUR_ORG/YOUR_REPO.git
git branch -M main
git push -u origin main
```

If the project already has a Git repository, do not run git init again. Instead, check the existing remote and push to the correct repository:

```
git remote -v
git status
git add .
git commit -m "Prepare deployment documentation and no-index safeguards"
git push
```

Part 12 - What Not to Commit to Git

- .env or any file containing real database passwords, app keys, mail passwords, API keys, or server credentials.
- node_modules because it can be regenerated with npm install.
- vendor unless you have a special deployment reason to store Composer dependencies in Git.
- public/build unless your team intentionally commits built frontend assets for shared hosting deployment.
- Large database dumps, local backups, temporary helper scripts, or private Excel data unless explicitly approved.

Keep .env.example in Git so other developers know which environment variables are needed, but do not put real production secrets in it.

Part 13 - Deployment Checklist

- Frontend assets built with npm run build.
- Subdomain created in z.com/cPanel.
- Subdomain document root points to the copied Laravel public files.
- Private Laravel app folder contains the rest of the application.
- public/index.php paths point to the private app folder.
- .env uses production settings and z.com database credentials.
- Database imported into z.com MySQL.
- HTTPS is enabled.
- No-index safeguards are uploaded.

- Login page loads correctly at the subdomain.
- Git repository excludes secrets and generated folders.
- Latest approved code is committed and pushed.